Production-Ready MLOps Pipeline for Smartphone-Based Sprint Biomechanics: From Research Prototype to Scalable Coaching Platform

Y. Teja
Dept. of CSE (AI & ML)
Dayananda Sagar University
tejachey75@gmail.com

Shaik Sadiya Anmol
Dept. of CSE (AI & ML)
Dayananda Sagar University
anmolsadiya583@gmail.com

Varsha R
Dept. of CSE (AI & ML)
Dayananda Sagar University
varshareddy3422@gmail.com

Yeshitha P
Dept. of CSE (AI & ML)
Dayananda Sagar University
yeshithayeshitharaj@gmail.com

Amritha Abdul Salam
Dept. of CSE (AI & ML)
Dayananda Sagar University
amrithaabdul-aiml@dsu.edu.in

Abstract—Laboratory motion capture for sprint biomechanics costs USD 100k–500k, which puts it out of reach for most athletes. We present a reproducible, containerised MLOps pipeline that combines a single 60 fps smartphone video (MediaPipe Pose) with a Galaxy Watch4 Classic IMU (100 Hz) to extract 19 sprint features. Labels come from an objective, automatically measured quantity, the athlete's body-size-normalised running velocity, instead of subjective ratings. Video–IMU integration is validated quantitatively: stride frequency from the two sensors agrees with a mean absolute error of 0.07 Hz ($r = 0.95$), and automatic checks flagged two of nine watch files that did not belong to the filmed sprint. The pipeline is versioned (DVC, MLflow), gated by a model-promotion test, monitored with Prometheus/Grafana, and deployed on Docker and Kubernetes. A request takes a median 2.1 s; INT8 quantisation and pruning shrink the classifier from 53 KB to 19 KB (12 KB compressed) with 99.6–100% prediction agreement. Under 50 concurrent users an unprotected deployment was OOM-killed; adding admission control, load shedding and memory recycling kept all pods alive, with 0 failed requests in two rolling updates. In a leakage-free leave-one-subject-out pilot with eight athletes (nine trials), three-tier classification did not exceed chance (best 55.6%, $p = 0.16$); earlier reported accuracies came from label circularity and augmentation leakage, which we document.

Index Terms—sprint biomechanics, MLOps, pose estimation, wearable IMU, drift detection, model registry

I. Introduction

Sprint speed depends on stride frequency, ground contact time and joint kinematics, and measuring them lets coaches find inefficiencies and asymmetries [1]. Optical motion capture is accurate but expensive and confined to laboratories [2]. Markerless pose estimators such as MediaPipe/BlazePose [3] and OpenPose [4] run on commodity hardware and single-camera video can approximate laboratory gait measures [5], while consumer smartwatches add inertial signals. Turning such a prototype into a service raises engineering questions that accuracy alone does not answer: are the two sensors really synchronised, are results reproducible, is a retrained model safe to deploy, and what happens when the data change [6]?

This paper reports an MLOps pipeline for smartphone video plus smartwatch IMU sprint analysis and, as importantly, an honest evaluation of it. Its contributions are:

1) an objective, annotation-free performance label (normalised running velocity) and a data-quality gate that quarantines mismatched sensor files automatically (Sections III–IV);
2) a versioned, containerised pipeline with experiment tracking, a promotion-gated model registry, a drift monitor and a retraining loop (Section V);
3) a leakage-free, subject-independent evaluation that corrects accuracies reported in an earlier version of this work (Section VI);
4) measured latency, throughput, fault-tolerance and drift-detection evidence (Sections VII–IX).

II. Related Work

Markerless motion analysis. Vision-based systems are replacing marker-based capture in sport [2]; lightweight estimators such as BlazePose [3] and OpenPose [4] run in real time on commodity hardware, and single-camera deep-learning pipelines recover clinically relevant gait parameters [5]. Most such studies examine walking, not maximal-effort sprinting, where motion blur and speed stress the estimators. Sprint biomechanics. Top running speed is set by the force applied to the ground rather than by leg-swing rate [1], which is why ground contact time and step frequency are the key features here and why a wrist-worn IMU, which senses arm swing and trunk impact, is only an indirect proxy. MLOps and monitoring. Machine-learning systems accumulate hidden technical debt in data dependencies, configuration and feedback loops [6]; experiment tracking and model registries [11] and concept-drift handling [14] are standard remedies, but

they are rarely evaluated on small, sensor-heterogeneous sports data, which is the gap this paper addresses.

III. Data and Labels

A. Dataset

Nine maximal-effort fly sprints were recorded from eight athletes (one athlete ran two trials) in a single outdoor session on a synthetic track. All participants gave informed consent, and participant details are confidential and not reported. Video was recorded with a tripod-mounted smartphone at 1920×1080, 60 fps (HEVC), camera orthogonal to the running direction; athletes crossed the field of view in 0.9–1.7 s. A Samsung Galaxy Watch4 Classic worn on the left wrist recorded triaxial acceleration and angular velocity at a measured 99.3–99.6 Hz with the Sensor Logger application. The raw recordings cannot be released because of the consent given; the code, parameters, tests and container definition are public.[1] The dataset is small: eight athletes and nine trials limit every claim in this paper, and we therefore present the classification study as a pilot (Section XII).

B. Objective performance labels

An earlier version of this work labelled athletes by clustering the same features later used for classification, which is circular, and described the labels as expert ratings. We replace them with a measured quantity that is not a model input: the athlete's horizontal velocity. The mid-hip landmark position is regressed on time with a Theil–Sen estimator [7] ($R^2 \geq 0.998$ for all nine recordings), and the pixel velocity is divided by a posture-invariant body length L (trunk + thigh + shank, median over frames), which equals 0.779× stature [8]. This gives velocity in statures per second, independent of camera distance (verified in unit tests at 120 and 300 px stature). Assuming a 1.70 m stature, velocities range from 6.7 to 9.4 m/s (mean 7.7). Tiers are cohort tertiles (Beginner / Intermediate / Elite); both trials of the repeated athlete fall in the same tier (Fig. 1). Labels are measured automatically, so new recordings need no annotator. Only frames of the sprinting athlete are used: MediaPipe occasionally locks onto a static false detection, and we keep the continuous track with the largest horizontal travel.

C. Features

Nineteen features are extracted per recording: nine from video (mean knee and hip angles, knee and hip symmetry indices $|X_R - X_L|/\bar{X} \times 100\%$, trunk lean, vertical hip oscillation in statures, step frequency, ground contact time (GCT) and duty factor) and ten from the IMU (peak, mean and SD of acceleration and angular-velocity magnitude, RMS jerk, arm-swing frequency from a Welch PSD [9], and SPARC smoothness [10]). A planted foot is stationary in the world frame, so stance is detected when

[1] https://github.com/TejaChey/Multimodal_AI_Based_Motion_Analysis

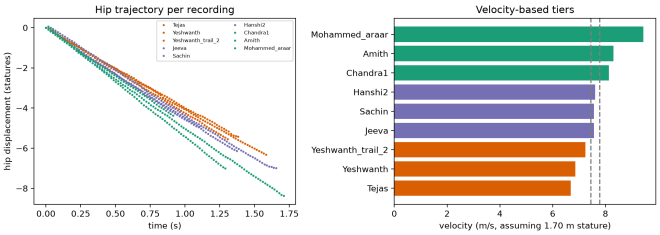


Fig. 1. Velocity labels. Left: hip trajectories (statures) per recording; right: measured velocity and tertile thresholds.

TABLE I
Recordings, measured velocity v and automatic quality gate. Athletes are anonymised. Tier: B/I/E = Beginner/Intermediate/Elite (thresholds 7.45 and 7.79 m/s; 1.70 m stature assumed).

Rec.	v (m/s)	Tier	Step (Hz)	GCT (ms)	Gate
A5	9.44	E	4.59	70	pass
A1	8.30	E	3.98	103	pass
A2	8.14	E	3.94	89	fail
A3	7.61	I	4.07	98	pass
A6	7.56	I	3.82	101	pass
A4	7.56	I	3.24	117	fail
A8 trial 2	7.24	B	4.26	92	pass
A8 trial 1	6.86	B	4.24	112	pass
A7	6.68	B	3.78	85	pass

the slower ankle's horizontal speed falls below $0.4\,v$ (v = running speed); this threshold yields physiological step rates (3.2–4.6 Hz) and GCT (70–117 ms) on all clips and was not tuned on accuracy. Stride length, clip duration and frame count are excluded: the first is algebraically velocity over step rate (label leakage) and the others depend on how a clip was trimmed.

IV. Video–IMU Integration

The phone and watch clocks are independent. The IMU segment used is the highest-intensity window whose length equals the time the runner is tracked in the video; accelerometer and gyroscope streams are merged by nearest timestamp (20 ms tolerance), and fusion is early (feature-level concatenation after per-fold standardisation). Integration is validated rather than assumed (Fig. 2): (i) clock consistency: for 7 of 9 recordings the video creation time follows the watch's last sample by 1.6–5.7 s (median 2.4 s), whereas two files differ by −102 s and −422 s, i.e. they were recorded during another run; (ii) cadence agreement on the seven consistent recordings: video wrist-swing stride frequency versus the watch gyroscope gives MAE 0.073 Hz, bias −0.05 Hz, $r = 0.95$; (iii) waveform agreement: the cross-correlation between video wrist velocity and the dominant gyroscope axis has $|r|$ of 0.51–0.94 (median 0.76). These checks run automatically as a data-quality gate (Section V), which quarantined both mismatched files without manual inspection.

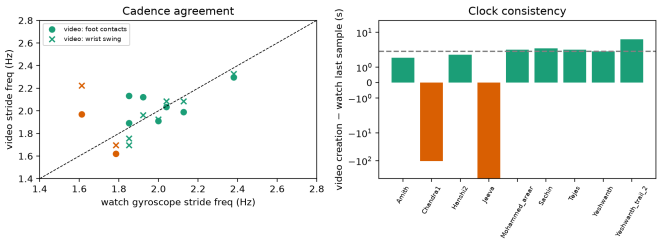


Fig. 2. Video–watch integration. Left: stride frequency from video vs. watch gyroscope (orange: mismatched files). Right: video–watch clock offset per recording.

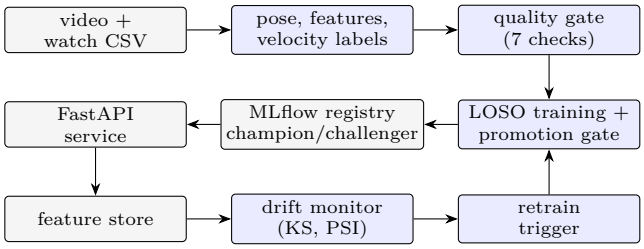


Fig. 3. Pipeline. Blue: DVC-versioned batch stages and monitoring; the service serves only the registry champion.

V. MLOps System

Versioning and tracking. A DVC pipeline (pose → features → quality gate → training) records a hash of the raw data; every MLflow [11] run logs the git commit, data hash, parameters, metrics and evaluation report. All settings live in one parameter file. Data-quality gate. Seven checks quarantine a recording instead of silently training on it: frame rate ($\geq$ 50 fps, because phones record variable 58–60 fps and 30 fps footage makes contact features unmeasurable), IMU rate (90–110 Hz), runner-track fit ($R^2 \geq 0.99$), tracked frames ($\geq$ 40), finite features, video–IMU cadence agreement ($\leq$ 0.3 Hz) and cohort clock consistency ($\leq$ 5 s from the median). The thresholds were set after inspecting these nine recordings and need re-validation on new data. The gate quarantines athlete A2 (cadence 0.48 Hz off, clock deviation −104 s) and A4 (clock deviation −424 s); seven recordings (six athletes) remain. Promotion gate. Six candidates (video, IMU or fused features; logistic regression or random forest, with augmentation) are evaluated by leave-one-subject-out (LOSO) cross-validation; the best by macro-F1 is the challenger. A challenger is promoted only if its permutation p-value is below 0.05 and its macro-F1 is not below the champion's. The p-value uses the maximum statistic over all candidates, which accounts for the selection [12]. The first model is registered as provisional so the service has something to serve, and the API labels it as such. The gate is deterministic: (1) if the training set has fewer than two recordings in a tier or fewer than five athletes, no model is trained and the reason is logged; (2) otherwise the best candidate's LOSO macro-F1 and max-statistic permutation p are computed; (3) with no champion, the model is registered as provisional; (4) with a champion, the challenger is refused if $p \geq 0.05$ or its macro-F1 is below the champion's; (5) every outcome is logged to MLflow and the retraining log. Its known limitation is that it compares macro-F1 across models evaluated on different cohorts, which is only approximately like-for-like.

TABLE II
Automatic data-quality gate: rule, observed range over the nine recordings, and failures.

Check	Rule	Observed	Fail
Video frame rate (fps)	≥ 50	58.3–59.7	0
IMU rate (Hz)	90–110	99.3–99.6	0
Runner-track fit R^2	≥ 0.99	0.998–1.000	0
Tracked frames	≥ 40	55–103	0
Features finite	all 19	all	0
Video–IMU cadence diff. (Hz)	≤ 0.3	0.001–0.479	1
Clock offset dev. from median (s)	≤ 5	−424–3.6	2

Serving and protection. A FastAPI service (/predict, /health, /metrics, /drift) in a Docker image loads the champion, hot-reloads it on promotion, returns the measured velocity as the primary result and the model's tier flagged as provisional, and logs every request to a feature store. Load testing exposed three failure modes that we fixed: (i) unbounded concurrency exhausted memory, since each pose pipeline needs about 0.6 GiB, so each process now admits a fixed number of pipelines and sheds the rest with a fast 503 and Retry-After; (ii) health checks queued behind predictions, so /health is served on the event loop; (iii) resident memory grew about 12 MiB per request, which returning freed heap to the OS reduced to about 3 MiB, and a safety net now drains and gracefully restarts a process above a memory threshold. Monitoring. Prometheus scrapes per-stage latency histograms, outcome counters, in-flight requests and model version (aggregated across worker processes) from the API, and drift and retraining metrics from a worker container; a provisioned Grafana dashboard (Fig. 6) displays them, and the worker can post alerts to a Slack-compatible webhook (tested against a mock endpoint). Kubernetes. The service runs as a Deployment with readiness/liveness probes, a rolling-update strategy with no unavailable replica, and a CPU-based horizontal autoscaler (2–5 pods), tested on a local single-node kind cluster. The Streamlit app is a thin client of the API. Tests. 43 automated tests (about 9 s) cover velocity recovery, leakage, each quality rule, drift false-alarm rate and power, the promotion gate, registry restore and the API contract; a regression test checks that the package reproduces the validated features to 10^{-6}.

VI. Classification Results

A. Protocol

Evaluation is subject-independent: LOSO cross-validation (eight folds; both trials of the repeated athlete

TABLE III
LOSO results (chance 33%). Best model per modality by macro-F1. p: permutation test (not selection-corrected).

Modality	Model	Acc. (%)	P	R	F1	95% CI / p
All recordings: 8 athletes, 9 trials						
Video	LogReg+aug	33.3	.33	.33	.32	9–75 / .44
IMU	LogReg+aug	55.6	.42	.56	.48	25–88 / .16
Fused	MLP	28.9±16.9	.23	.29	.24	13–89 / –
Watch files verified: 6 athletes, 7 trials						
Video	SVM+aug	28.6	.33	.33	.33	0–67 / .44
IMU	RF+aug	57.1	.50	.56	.52	25–86 / .10
Fused	RF+aug	14.3	.08	.11	.10	0–33 / .70

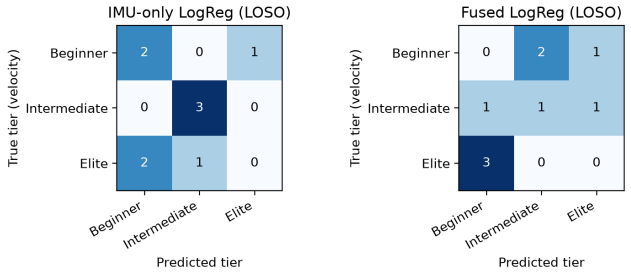


Fig. 4. Pooled LOSO confusion matrices (9 recordings): IMU-only logistic regression (left) and fused logistic regression (right).

are always held out together). In each fold, only on the training athletes, a standard scaler is fitted and the training set is optionally augmented to 300 samples per class (Gaussian jitter at 5% of the feature SD plus same-class interpolation). The held-out athlete therefore never influences scaling, augmentation or early stopping; a unit test confirms that a held-out athlete's prediction does not change with its own label. Classifiers are logistic regression ($C = 1$), an RBF-SVM ($C = 1$), a random forest (100 trees, depth 5) and an MLP (128–64–32 ReLU, batch normalisation, dropout 0.4/0.3/0.2, L2 10^{-3}, Adam 10^{-3}, at most 100 epochs, early stopping with patience 10; five seeds). We report accuracy, macro precision, recall and F1 over the nine pooled out-of-fold predictions, subject-level bootstrap 95% CIs (2000 resamples) and label-permutation p-values (200 permutations).

B. Results

Table III and Fig. 4 give the leakage-free results. No configuration is significantly above chance (smallest $p = 0.10$), several score below chance (a known artefact of leave-one-out with classes of three samples), and early fusion does not outperform the single modalities. A LOSO ridge regression of velocity does not beat a training-mean predictor either (MAE 0.80–0.88 vs. 0.71 m/s). The model deployed by the pipeline, selected among six candidates on seven verified recordings with selection-corrected p, scores 28.6% (macro-F1 0.24, $p = 0.67$) and is therefore served as provisional. Descriptively, GCT correlates negatively with velocity ($r = -0.52$) and step frequency positively ($r = 0.40$), consistent with [1], but $n = 9$ is too small for inference.

Relation to earlier accuracies. The earlier version of this system reported 82–100%. Those figures came from (a) K-Means labels computed on the input features themselves and (b) synthetic samples generated from all athletes before the train/test split, so every test athlete had near-copies in the training set. We report this because both pitfalls are common in small-sample sports analytics.

TABLE IV
Measured per-stage latency, median (p95 where given). Apple M5, MediaPipe 0.10.9. †Deployed configuration (pose complexity 1).

Stage	All cores	E-cores only
Video decode, per frame	1.4 ms	10.6 ms
Pose lite, per frame	7.1 (13.0) ms	45.8 (75.8) ms
Pose full†, per frame	11.5 (17.9) ms	61.3 (114) ms
Pose heavy, per frame	49.3 (56.3) ms	212 (273) ms
Features, per sprint	8.1 ms	49.9 ms
MLP, Keras predict	14.5 ms	95 ms
MLP, TFLite (52 KB)	1.1 μs	9.3 μs
End-to-end, per clip†	2.1 s	12.7 s

TABLE V
TFLite variants of the MLP (19–128–64–32–3). Latency: P-cores / E-cores, 1 thread, laptop CPU (not a phone). Agreement: fraction of predictions equal to the FP32 Keras model on 2007 points.

Variant	KB	gzip KB	μs (P/E)	Agree.
FP32	53.4	48.6	1.2/9.8	100%
INT8 dynamic	19.1	15.6	1.2/7.5	100%
INT8 full-integer	22.2	16.3	1.2/7.0	99.85%
Pruned 50%+INT8	19.1	11.9	1.2/7.7	99.6%

VII. Latency

Pose estimation dominates (Table IV); once exported to TFLite the classifier costs microseconds, and the Keras predict figure (14.5–95 ms) is framework overhead, not model compute. Through the Docker API a full request (upload, pose, features, classifier) has a median of 2.1 s for one user. On-phone execution was not measured and is future work.

Quantisation cuts the model by 64% (to 22% of its size after pruning and compression) with at most 0.4% prediction disagreement; latency is already microseconds, so compression matters for download size, not speed. The earlier version's 12 MB → 1.8 MB figures cannot apply: the model was never larger than 53 KB. Fidelity to FP32 is not task accuracy, which remains at chance level.

VIII. Drift Monitoring and Retraining

Thresholds. Each feature is tested against the training reference with a two-sample Kolmogorov–Smirnov test [13] at the Bonferroni-corrected $\alpha = 0.05/19 = 0.0026$, which bounds the family-wise false-alarm rate at 5% (a unit test asserts $\leq 10\%$ over 300 null windows); the population stability index is a secondary signal [14]. A Monte-Carlo

power analysis (Fig. 5) shows that a 1-SD shift in one feature is detected with 84% probability only with a reference of ≥ 100 and a window of ≥ 30; with our 9-sample reference power never exceeds 29%. Alarms are therefore armed only above those sizes and reported as informational below them; our reference has 7 recordings, so no alarm is armed today. Retraining is triggered by an armed alarm, every three new quality-passed recordings (a parameter; low-risk because of the promotion gate), or manually.

Device-shift replay. Seven real clips were replayed through the API with simulated device changes. Unmodified clips raise no alarm; a 50 Hz watch is rejected by the quality gate in 7/7 cases and never reaches the monitor; a $\times 1.5$ accelerometer gain raises an informational alarm on peak acceleration, acceleration SD and RMS jerk ($p = 0.00058$ each). A 30 fps camera makes contact features unmeasurable, so 60 fps capture is a hard requirement. Changing population. Replaying the nine recordings in capture order from a cold start, two are quarantined, a first (provisional, $p = 0.099$) model appears at the 8th arrival, and the retrained challenger at the 9th is refused ($p = 0.65$, macro-F1 0.24 vs. 0.56). The loop and gate work, but an accuracy benefit from retraining could not be demonstrated at this sample size.

IX. Throughput, Scalability and Availability

Single Docker node. With a real 2–3 s clip (including upload) one user sees a median 2.1 s; throughput saturates near 1.2 clips/s (≈70/min) on a 10-core node irrespective of worker count, because MediaPipe already parallelises within a request (214 requests, 0 failures). Killing one of two workers caused no failed health probe (replacement in 0.03 s), and a server-process crash was recovered by the restart policy in 0.9–1.0 s. Kubernetes under overload. With 50 concurrent users, a single node cannot give interactive latency, so what matters is that the system degrades safely. In our first attempt, with unbounded concurrency, every pod was OOM-killed and 13 534 of 13 535 requests failed. With the protections of Section V and the autoscaler (CPU target 70%), a 300 s run with 50 users gave: replicas $2 \to 4$ after 50 s and $\to 5$ after 170 s; of 778 requests, 238 were served (median 21.7 s including queueing, p95 23.8 s, 0.79 clips/s), 515 were shed with a 503 and 25 (3%) failed with a connection error; no pod was OOM-killed, and two were gracefully recycled above the memory threshold. Throughput did not scale with pods because all pods share one node's CPU, and multi-node scaling was not tested. Faults. Under continuous traffic on two replicas, two rolling updates produced 0 failed health probes (of 764) and 0 failed predictions (of 20), taking 8.1 s; two forced pod deletions produced 0 failed probes (of 611) and 1 failed prediction (of 15, the request in flight on the killed pod), with 4.0–4.6 s to restore two ready replicas. Availability. A fault-free health probe succeeded 60/60 times over five minutes; a probe that runs until submission is under way and its measured window will be reported as is. A short window cannot support a figure near 99.5%, and none is claimed.

TABLE VI
Reproducibility settings.

Camera	smartphone, 1080p HEVC, 60 fps, tripod
Watch	Galaxy Watch4 Classic (left wrist), 99.3–99.6 Hz
Compute	Apple M5 (10 cores), 16 GB, macOS 27.0.1; Docker VM 10 CPUs, 8 GB
Software	Python 3.11.17, MediaPipe 0.10.9, scikit-learn 1.7.2, NumPy 1.26.4, pandas 2.3.3, SciPy 1.17.1, OpenCV 4.11, TensorFlow 2.18.1, MLflow 3.5.1, DVC 3.67.1, FastAPI 0.162.2, Prometheus 2.54.1, Grafana 11.2.0, kind 0.33 (Kubernetes), Locust 2.47
Settings	seed 42 (MLP 42–46); pose complexity 1; all in params.yaml

X. Reproducibility

Table VI lists the environment. dvc repro rebuilds the pipeline from raw recordings, every number here is regenerated by the released scripts, and the tests run without the recordings. Pose features differ slightly between macOS and the Linux container (e.g. 8.30 vs. 8.32 m/s), so the platform is stated with each result.

XI. Discussion

What the data can and cannot say. The measurement layer is validated: velocity labels fit every trajectory almost perfectly, video and watch agree on cadence, and mismatched files are caught automatically. The prediction layer is not: with eight athletes, the standardisation of many weakly informative features lets a classifier fit spurious patterns in small training folds, which is the same effect that makes fusion worse than a single modality in Table III. For this reason the service reports the measured velocity as its primary output and the model tier only as provisional. Earlier UTD-MHAD experiment. The earlier version reported 87% three-class fusion accuracy on UTD-MHAD jogging, with tiers defined as terciles of a video feature (mean hip angle). Re-running that experiment gives 62.5% and, because the label is derived from an input, it is not a valid skill test; we withdraw it. As a measurement check we also compared video and thigh-IMU stride frequency on its 32 jogging-in-place trials and found no agreement (MAE 0.54 Hz, $r = -0.02$), so we make no claim that our cadence method transfers to that activity.

XII. Limitations

(1) Eight athletes from one session: the classification study is a pilot and supports no claim of generalisation. (2) Tiers are cohort-relative tertiles; absolute velocity assumes a 1.70 m stature (≈ ±5% per 8.5 cm). (3) 2-D pose from one side view; joint-angle accuracy was

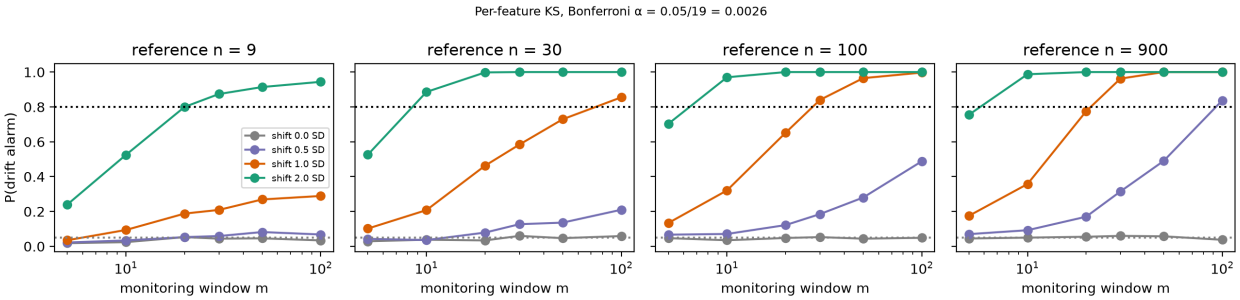


Fig. 5. Detection probability of the per-feature KS monitor versus window size for reference sets of 9, 30, 100 and 900 samples.

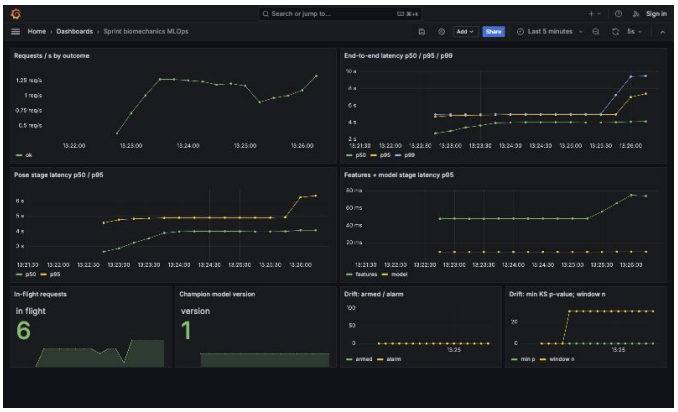


Fig. 6. Grafana dashboard during a load test: request rate, end-to-end and per-stage latency percentiles, in-flight requests, model version and drift state.

not validated against motion capture. (4) Quality-gate thresholds were set on these recordings. (5) Latency and compression were measured on a laptop CPU, not on phones; Kubernetes was tested on one node, so multi-node scaling and long-run availability are unmeasured. (6) We did not use a dedicated feature-store product, a data-validation framework or hyper-parameter search; a SQLite store and the built-in quality gate fill those roles, and tuning on eight athletes would only fit noise. (7) Retraining benefit could not be demonstrated.

XIII. Conclusion

We presented a reproducible smartphone-plus-smartwatch sprint-analysis pipeline with an objective velocity label, a validated and automatically checked video–IMU integration, a promotion-gated registry, drift monitoring justified by a power analysis, and measured latency, throughput and fault-tolerance. A leakage-free pilot showed that three-tier skill classification is not yet reliable with eight athletes, and that earlier accuracies were artefacts of label circularity and augmentation leakage. At least 100 labelled recordings are needed before drift alarms can be armed; because labelling is automatic, routine use can reach that milestone cheaply, which is the next step.

Acknowledgment

The authors thank the Department of Physical Education at Dayananda Sagar University for facility access and athlete recruitment support, and all participating athletes.

References

[1] P. G. Weyand, D. B. Sternlight, M. J. Bellizzi, and S. Wright, "Faster top running speeds are achieved with greater ground forces not more rapid leg movements," J. Appl. Physiol., vol. 89, no. 5, pp. 1991–1999, 2000.

[2] S. L. Colyer, M. Evans, D. P. Cosker, and A. I. T. Salo, "A review of the evolution of vision-based motion analysis and the integration of advanced computer vision methods towards developing a markerless system," Sports Med. Open, vol. 4, art. 24, 2018.

[3] V. Bazarevsky et al., "BlazePose: On-device real-time body pose tracking," arXiv:2006.10204, 2020.

[4] Z. Cao, G. Hidalgo, T. Simon, S.-E. Wei, and Y. Sheikh, "OpenPose: Realtime multi-person 2D pose estimation using part affinity fields," IEEE Trans. Pattern Anal. Mach. Intell., vol. 43, no. 1, pp. 172–186, 2021.

[5] Ł. Kidziński et al., "Deep neural networks enable quantitative movement analysis using single-camera videos," Nat. Commun., vol. 11, art. 4054, 2020.

[6] D. Sculley et al., "Hidden technical debt in machine learning systems," in Proc. NeurIPS, 2015.

[7] P. K. Sen, "Estimates of the regression coefficient based on Kendall's tau," J. Am. Stat. Assoc., vol. 63, no. 324, pp. 1379–1389, 1968.

[8] R. Drillis and R. Contini, "Body segment parameters," New York Univ., School of Eng. and Science, Tech. Rep. 1166-03, 1966.

[9] P. D. Welch, "The use of fast Fourier transform for the estimation of power spectra," IEEE Trans. Audio Electroacoust., vol. 15, no. 2, pp. 70–73, 1967.

[10] S. Balasubramanian, A. Melendez-Calderon, A. Roby-Brami, and E. Burdet, "On the analysis of movement smoothness," J. NeuroEng. Rehabil., vol. 12, art. 112, 2015.

[11] M. Zaharia et al., "Accelerating the machine learning lifecycle with MLflow," IEEE Data Eng. Bull., vol. 41, no. 4, pp. 39–45, 2018.

[12] B. Phipson and G. K. Smyth, "Permutation p-values should never be zero," Stat. Appl. Genet. Mol. Biol., vol. 9, no. 1, art. 39, 2010.

[13] F. J. Massey, "The Kolmogorov–Smirnov test for goodness of fit," J. Am. Stat. Assoc., vol. 46, no. 253, pp. 68–78, 1951.

[14] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," ACM Comput. Surv., vol. 46, no. 4, art. 44, 2014.